

02

깃으로 버전 관리하기

02-1 깃 저장소 만들기

02-2 버전 만들기

02-3 커밋 내용 확인하기

02-4 버전 만드는 단계마다 파일 상태 알아보기

02-5 작업 되돌리기



깃 저장소 만들기 - (1)

1. 깃 저장소를 만들기 전에 터미널 창을 열고 다음과 같이 입력해서 홈 디렉터리에 hello-git이라는 디렉터를 만들고 그리고 cd 명령을 사용해 hello-git 디렉터리로 이동

```
$ mkdir hello-git  
$ cd hello-git
```

2. hello-git 디렉터리 안의 내용을 살펴보기 위해 ls -la 명령을 입력
ls 명령 다음에 -l 옵션과 함께 -a라는 옵션을 붙인 것은 숨김 파일이나 디렉터리까지 포함해서 모두 보여달라는 의미

```
$ ls -la
```

3. 화면에 두 줄짜리 결과가 나타남
마침표가 하나인 항목(.)은 현재 디렉터를 나타내고, 마침표가 2개인 항목은(..)은 상위 디렉터를 나타냄
아직 아무 파일도 만들지 않았으므로 파일이 하나도 없음

깃 저장소 만들기 - (2)

- 이 디렉터리(hello-git)에 저장소를 만들기 위해 터미널에 다음과 같이 입력
현재 디렉터리에서 깃을 사용할 수 있도록 초기화

```
$ git init
```

- 'Initialized empty Git repository...'라는 메시지가 나타난다면 이제부터 hello-git에서 깃을 사용할 수 있음
터미널에 있는 파일 경로 끝에 (main)이라고 표시되는데 hello-git 디렉터리에는 깃을 위한 저장소가 생겼다는 의미
- ls 명령을 사용해서 다시 한번 디렉터리 안의 내용을 확인

```
$ ls -la
```

- 처음에 살펴봤을 때 내용과 비교하면 .git이라는 디렉터리가 생김
이 폴더는 깃을 사용하면서 버전이 저장될 '저장소(repository)'

깃에서 버전이란

- 버전은 문서를 수정하고 저장할 때마다 생기는 것
- 예를 들어 보고서를 작성할 때 '초안'이라는 이름으로 저장함
이미 만든 보고서를 수정하면서 수정 전 내용을 보관해야 할 경우가 있어 '수정'처럼 파일 이름을 바꿔서 저장
만약 1,000개가 넘는 문서의 수정 내용을 이런 방식으로 저장한다면 어찌어찌 몇 개의 파일 이름은 기억한다고 해도,
나중에는 어떤 파일에서 어떤 내용을 수정했는지 기억할 수 없음
- 이렇게 파일을 다른 이름으로 저장해 버전을 만드는 방법보다 훨씬 쉽게 버전을 만들고,
또한 버전을 만든 시간과 수정 내용까지 기록할 수 있는 것이 바로 깃과 같은 버전 관리 시스템
- 깃에서 버전을 관리하면 원래 파일 이름은 그대로 유지하면서 파일에서 무엇을 변경했는지를 변경 시점마다 저장할 수 있음
또, 버전마다 작업한 내용을 확인할 수 있고, 그 버전으로 되돌릴 수도 있음

스테이지와 커밋 이해하기 - (1)

작업 트리

- 작업 트리(working tree)는 파일 수정, 저장 등의 작업을 하는 디렉터리
- 작업 디렉터리(working directory)라고도 함
- 앞에서 만들었던 hello-git 디렉터리가 작업 트리
즉, 우리 눈에 보이는 디렉터리가 바로 작업 트리

스테이지

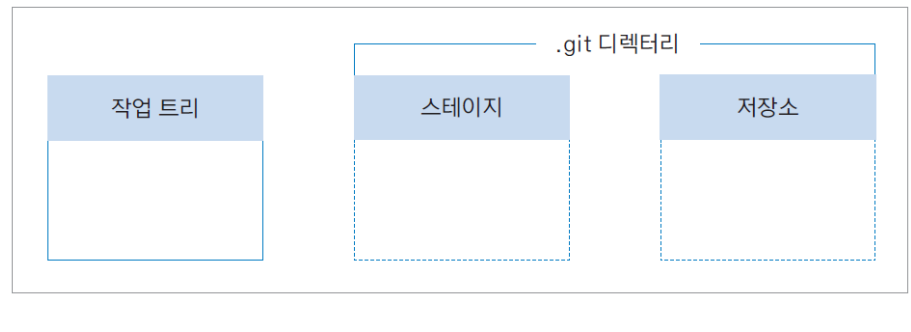
- 스테이지(stage)는 버전으로 만들 파일이 대기하는 곳
- 스테이징 영역(staging area)이라고도 함
- 예를 들어 작업 트리에서 파일 10개를 수정했는데 4개만 버전으로 만들려면
이 파일 4개만 스테이지로 넘겨주면 됨

저장소

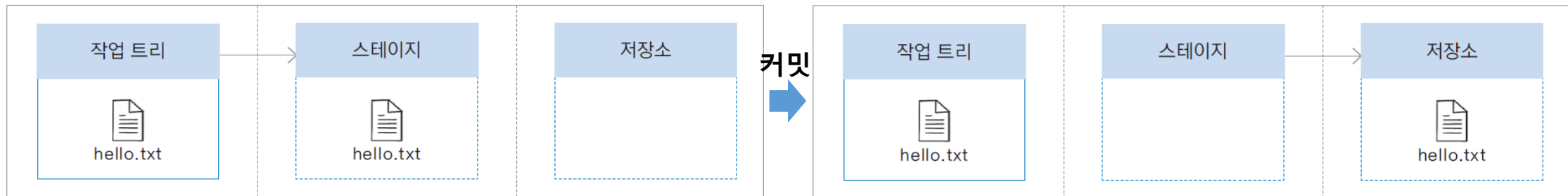
- 저장소, 즉 리포지토리(repository)는 스테이지에서 대기하고 있던 파일들을 버전으로 만들어 저장하는 곳

스테이지와 저장소는 눈에 보이지 않음

깃을 초기화했을 때 만들어지는 .git 디렉터리 안에 숨은 파일
형태로 존재하는 영역



스테이지와 커밋 이해하기 - (2)



- 예를 들어 hello.txt 파일 문서를 수정하고 저장하면 그 파일은 작업 트리에 있게 됨
수정한 hello.txt 파일을 스테이지에 넣고 다른 파일도 수정한 뒤 스테이지에 넣어 둠
파일 수정을 끝내고 스테이지에 다 넣었다면 버전을 만들기 위해 깃에게 '커밋(commit)' 명령을 내림
그러면 새로운 버전이 생성되면서 스테이지에 대기하던 파일이 모두 저장소에 저장됨

깃이 버전을 만드는 과정

1. 작업 트리에서 문서를 수정
2. 수정한 파일 가운데 버전으로 만들고 싶은 것을 스테이징 영역, 즉 스테이지에 저장
3. 스테이지에 있던 파일을 저장소로 커밋하면 버전이 만들어짐

작업 트리에서 문서 수정하기 - (1)

1. 앞에서 hello-git 디렉터리에서 깃을 초기화했으므로 이제부터 hello-git 디렉터리에서 버전 관리를 할 수 있음
앞의 내용을 따라왔다면 현재 hello-git 디렉터리로 이동해 있을 것이므로 먼저 깃 상태를 확인하기 위해 다음과 같이 입력

```
$ git status
```

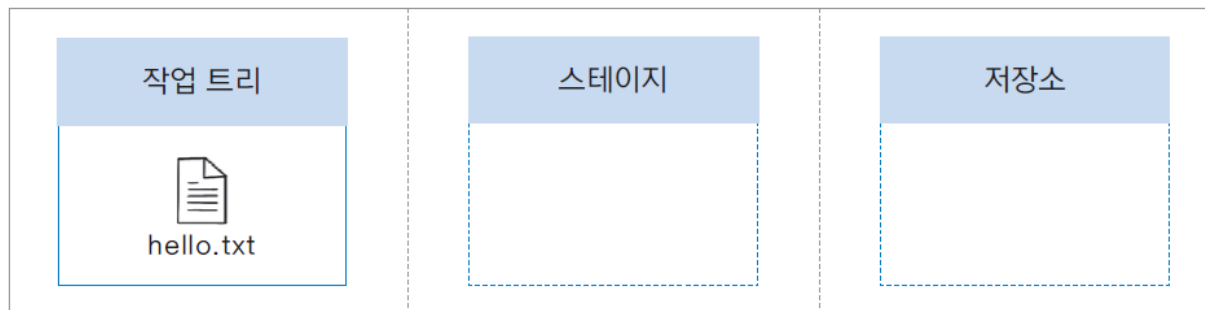
2. 깃의 상태를 보여 주는 메시지가 나타나는데 어떤 의미인지 간단히 살펴봄
On branch main: 현재 main 브랜치에 있음
No commits yet: 아직 커밋한 파일이 없음
nothing to commit: 현재 커밋할 파일이 없음
3. hello-git 디렉터리에서 터미널 창에 다음과 같이 입력해 hello.txt라는 파일을 만들

```
$ vim hello.txt
```

작업 트리에서 문서 수정하기 - (2)

4. vim 편집기가 열리면 `{ I }` 또는 `{ A }`를 눌러 입력 모드로 바꿈
간단하게 숫자 '1'을 입력하고 `{ Esc }`를 눌러 ex 모드로 바꾼 후 `:wq`를 입력하고 `{ Enter }`를 누름
문서가 저장되면서 편집기가 종료
5. 터미널 창으로 돌아와서 `ls -la` 명령을 입력
hello.txt 파일이 디렉터리 안에 만들어진 것을 보고 다음과 같이 입력해서 git의 상태를 다시 한번 확인
branch main에 hello.txt라는 untracked files가 있다는 메시지가 나타남
git에서는 버전을 아직 한 번도 관리하지 않은 파일을 untracked files라고 함

```
$ git status
```



수정한 파일을 스테이지에 올리기 — git add

1. 깃에서 스테이징할 때 사용하는 명령은 git add

터미널에 다음과 같이 입력해도 아무 내용도 나타나지 않을 것이지만 아무 일도 안 한 것은 아님

```
$ git add hello.txt
```

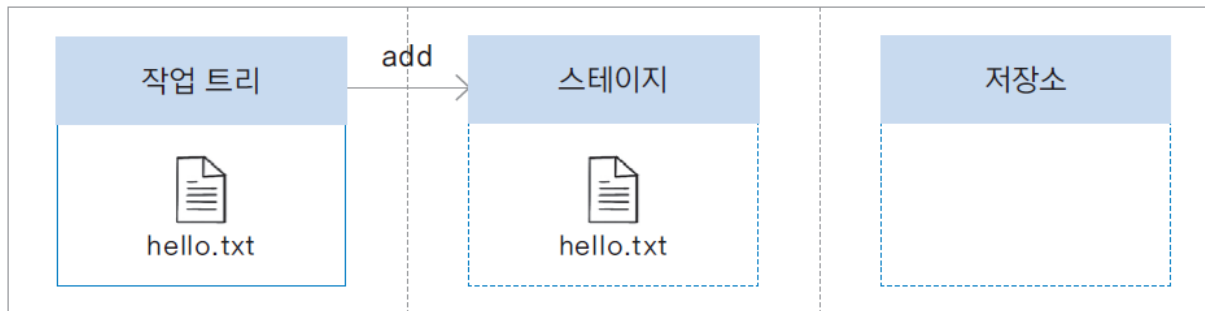
2. 무엇이 바뀌었는지 깃의 상태를 확인

```
$ git status
```

3. untracked files:이라는 문구가 changes to be committed:로 바뀌었음

그리고 hello.txt 파일 앞에 new file:이라는 수식어가 추가로 나타남

'새 파일 hello.txt를 (앞으로)커밋할 것이다.'라는 뜻



스태이징한 파일 커밋하기 — `git commit` - (1)

1. 깃에서 파일을 커밋하는 명령은 `git commit`
한 칸 띄운 후에 `-m` 옵션을 붙이고 커밋과 함께 저장할 메시지를 적음
이 메시지를 커밋 메시지라고 함

```
$ git commit -m "message1"
```

2. 커밋한 후에 결과 메시지를 보면 파일 1개가 변경되었고(1 file changed), 파일에 1개의 내용이 추가되었다(1 insertion(+))고 나타남
스테이지에 있던 `hello.txt` 파일이 저장소에 추가된 것
3. 커밋한 후 깃의 상태 확인

```
$ git status
```

4. 버전으로 만들 파일이 없고(nothing to commit), 작업 트리도 수정 사항 없이 깨끗하다(working tree clean)고 나타남

스테이징한 파일 커밋하기 — git commit - (2)

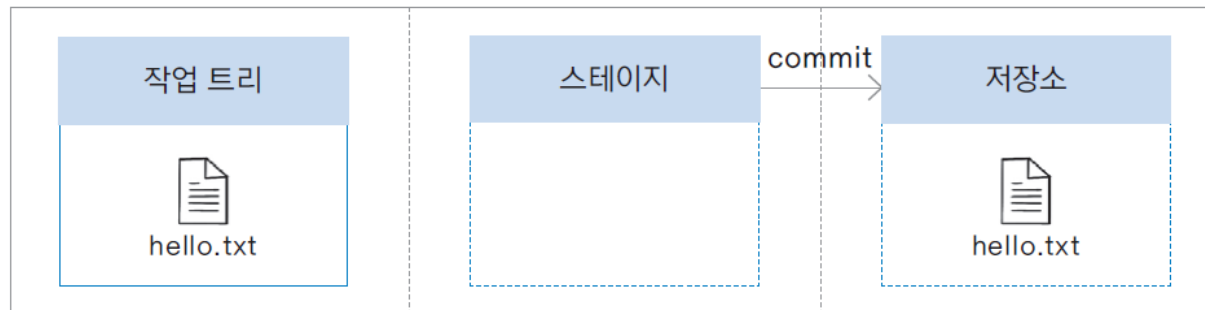
5. 저장소에 저장된 버전을 확인할 때는 log 명령을 사용

```
$ git log
```

6. 방금 커밋한 버전을 설명하는 정보가 나타남

커밋을 만든 사람과 시간, 커밋 메시지가 함께 보임

수정한 파일을 커밋하면 이렇게 수정과 관련된 여러 정보를 함께 저장할 수 있고 필요할 때 확인할 수도 있음



스테이징과 커밋 한꺼번에 처리하기 — `git commit -am` - (1)

1. 터미널에서 다음과 같이 입력해서 vim 편집기에 hello.txt 파일을 엽

```
$ vim hello.txt
```

2. vim이 열리면 `{ I }`나 `{ A }`를 눌러 입력 모드로 바꿈
그리고 숫자 '2'를 추가한 후 `{ Esc }` 누르고 `:wq`를 입력해 문서를 저장하면서 편집기를 종료
3. hello.txt 파일은 이전에 커밋한 적이 있으므로 `git commit` 명령에 `-am` 옵션을 붙여서 스테이징과 커밋을 한꺼번에 처리할 수 있음

```
$ git commit -am "message2"
```

4. 스테이징과 커밋 과정이 한꺼번에 보임

스테이징과 커밋 한꺼번에 처리하기 — `git commit -am - (2)`

5. 방금 커밋한 버전에 어떤 정보가 들어 있는지 확인

```
$ git log
```

6. hello.txt의 버전 2개가 나타나는데, 가장 최근 버전 정보가 'message2'라는 메시지와 함께 맨 위에 보임

커밋 기록 자세히 살펴보기 — git log

- 깃에서 자주 사용하는 명령으로, 커밋했던 기록을 살펴보는 git log가 있음
- git log 명령을 입력하면 지금까지 만든 버전이 화면에 보이고 버전마다 설명도 함께 나타남
- commit이라는 항목 옆에 영문과 숫자로 된 긴 문자열이 나타나는데 이것을 **커밋 해시(commit hash)**, 또는 **깃 해시(git hash)**라고 함
- 커밋을 구별하는 '아이디'라고 생각하면 쉬움
- 커밋 해시 옆에 있는 (HEAD -> main)는 이 버전이 가장 최신이라는 표시
- Author 항목에는 버전을 누가 만들었는지, Date 항목에는 버전이 언제 만들어졌는지 나타남
- 그 아래에는 작성자가 기록한 커밋 메시지가 나옴
- 이렇게 git log 명령을 입력했을 때 나오는 정보를 묶어 간단히 **커밋 로그**라고 함



```
MINGW64: c:/Users/funco/hello-git
funco@DESKTOP-CB5T406 MINGW64 ~/hello-git (main)
$ git log
commit a09e52ed26b4976269b97d942c63cd85bfc55e20 (HEAD -> main)
Author: kyunghee <jump2dev@gmail.com>
Date:   Fri May 13 14:53:36 2022 +0900
    message2

commit 250b27439bda6209caa214e947012a2883461107
Author: kyunghee <jump2dev@gmail.com>
Date:   Fri May 13 11:39:05 2022 +0900
    message1

funco@DESKTOP-CB5T406 MINGW64 ~/hello-git (main)
$
```

변경 사항 확인하기 — git diff – (1)

1. 앞의 내용을 계속 따라왔다면 hello.txt 파일에는 숫자 1부터 2까지 입력되어 있고, 저장소에는 2개의 버전이 저장되어 있음
빔에서 hello.txt 파일을 열고 기존 내용 중에서 '2'를 지우고 'two'를 추가한 후 저장

```
$ vim hello.txt
```

2. git status 명령을 사용해 깃의 상태를 확인해 보면 hello.txt 파일이 수정되었고, 아직 스테이징 상태가 아니라고 나옴

```
$ git status
```

3. 방금 수정한 hello.txt 파일이 저장소에 있는 최신 버전의 hello.txt와 어떻게 다른지 확인하기 위해 git diff 명령을 사용

```
$ git diff
```

변경 사항 확인하기 — `git diff` - (2)

4. '-2'는 최신 버전과 비교할 때 hello.txt 파일에서 '2'가 삭제되었다는 뜻
'+two'는 hello.txt 파일에 'two'라는 내용이 추가되었다는 뜻
작업 트리에서 수정한 파일과 최신 버전을 비교한 후 수정한 내용으로 다시 버전을 만들려면 스테이지에 올린 후 커밋하고,
수정한 내용을 버리려면 `git restore` 명령을 사용해 취소
5. 이어지는 실습을 위해 hello.txt를 원래대로 되돌려 놓아야 함
빔에서 hello.txt 파일을 열고 'two' 부분을 숫자 '2'로 수정한 후 저장

tracked 파일과 untracked 파일 - (1)

1. vim에서 hello.txt 파일을 열고 숫자 '3'을 추가한 후 저장

```
$ vim hello.txt
```

2. 다음과 같이 입력해서 vim에 hello2.txt라는 새로운 파일을 만듦

```
$ vim hello2.txt
```

3. 새로 만든 hello2.txt에 알파벳 a, b, c, d를 한 줄에 한 글자씩 입력하고 파일을 저장한 후 편집기를 종료

4. hello.txt 파일과 hello2.txt 파일 모두 작업 트리에 있음

git status 명령을 사용해 어떤 상태인지 확인

```
$ git status
```

tracked 파일과 untracked 파일 - (2)

5. 앞에서 커밋했던 hello.txt 파일은 'Changes not staged for commit:'이라고 되어 있음
변경된 파일이 아직 스테이지에 올라가지 않았다는 뜻
그리고 파일 이름 앞에 modified: 라고 되어 있어 hello.txt가 수정되었다는 것을 알 수 있음
깃은 한 번이라도 커밋한 파일은 계속해서 수정 사항이 있는지 추적
깃이 추적하고 있다는 뜻에서 tracked 파일이라고 함
반면에 hello2.txt 파일 앞에는 아무것도 없고 바로 위에는 'untracked files:'이라고 되어 있음
hello2.txt 파일은 한 번도 커밋하지 않았으므로 수정 내역을 추적하지 않음
그래서 untracked 파일이라고 표시
6. 수정한 hello.txt 파일과 수정하지 않은 hello2.txt 파일 모두 git add 명령을 사용해서 스테이지에 올릴 수 있음

```
$ git add hello.txt  
$ git add hello2.txt
```

tracked 파일과 untracked 파일 - (3)

7. git status를 사용해 상태를 확인

마지막 버전 이후에 수정한 hello.txt는 modified:라고 표시되고, 한 번도 버전 관리를 하지 않은 hello2.txt는 new file:로 표시됨

8. 이 커밋에는 hello.txt를 수정한 내용과 새로 만든 hello2.txt 내용이 다 포함

커밋 메시지는 다음과 같이 작성하고 커밋이 성공적으로 되었다면 로그를 확인

```
$ git commit -m "message3"
$ git log
```

9. 'message3'라는 메시지를 붙인 커밋이 보이는데 각 커밋에 어떤 파일이 관련되었는지 알 수 없음

10. 커밋과 관련된 파일까지 함께 살펴보려면 git log 명령에 --stat 옵션을 사용

```
$ git log --stat
```

11. 가장 최근 커밋부터 순서대로 커밋 메시지와 관련 파일이 나열됨

message3 커밋은 hello.txt, hello2.txt 파일과 관련되어 있고, message2는 hello.txt 파일과 관련되었다는 것을 알 수 있음

unmodified, modified, staged 상태 – (1)

1. ls -la 명령을 사용해 hello-git 디렉터를 살펴보면 앞에서 버전을 저장한 hello.txt와 hello2.txt 파일이 있음
2. git status 명령을 사용해 git의 상태와 파일의 상태를 확인하면 작업 트리에 아무 변경 사항도 없음
'working tree clean'이라고 나타나면 현재 작업 트리에 있는 모든 파일의 상태는 unmodified라는 뜻
3. hello2.txt에서 a만 남기고 나머지 내용을 삭제한 후 파일을 저장하고 편집기를 종료

```
$ vim hello2.txt
```

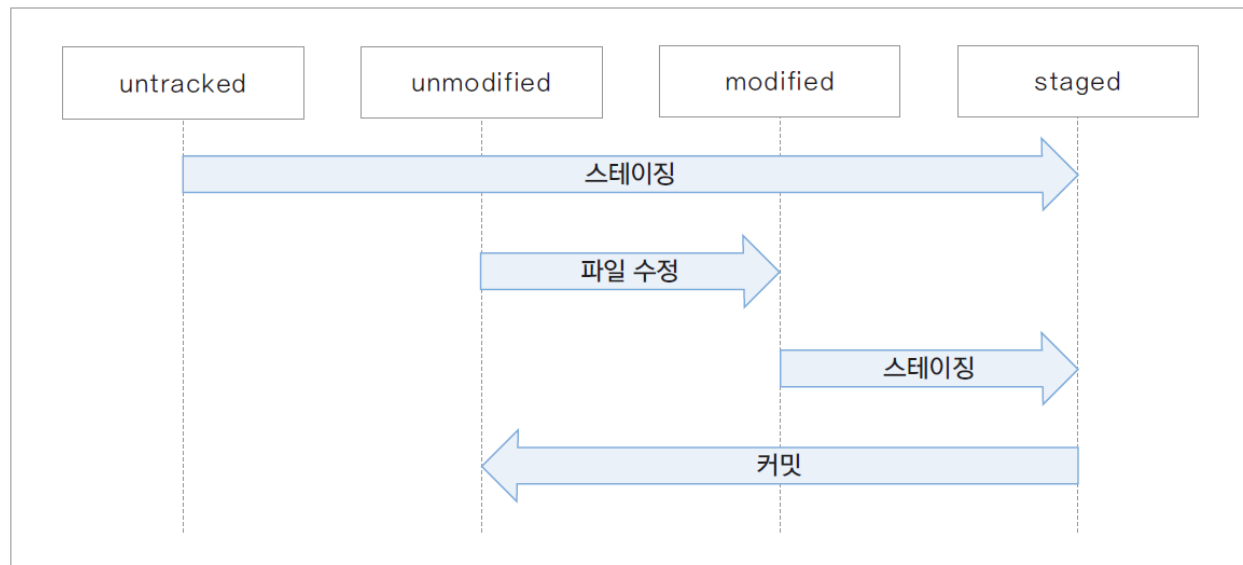
4. git status 명령을 실행하면 hello2.txt 파일이 수정되었고 아직 스테이지에 올라가지 않았다고 나타남
'Changes not stage for commit'라는 메시지가 나타나면 파일이 수정만 된 modified 상태를 뜻함
5. git add 명령을 사용해 스테이지에 올리고 git status 명령을 실행해 보세요. 커밋할 변경사항이 있다고 함
'Changes to be committed:'라는 메시지가 나타나면 커밋 직전 단계, 즉 staged 상태

```
$ git add hello2.txt  
$ git status
```

unmodified, modified, staged 상태 – (2)

6. 스테이지에 있는 hello2.txt 파일을 커밋하고 git status 명령을 실행
커밋을 끝내고 나면 hello2.txt 파일의 상태는 수정하기 직전인 unmodified 상태로 돌아감

```
$ git commit -m "delete b, c, d"  
$ git status
```



작업 트리에서 수정한 파일 되돌리기 — git restore

1. 먼저 vim 편집기에서 hello.txt 파일을 열고 숫자 '3'을 'three'로 수정한 후 저장하고 편집기를 종료

```
$ vim hello.txt
```

2. git status 명령을 사용해 보면 hello.txt가 수정되었지만 아직 스테이지에 올라가 있지 않음
그리고 두 번째 괄호 안의 메시지를 보면, 작업 디렉터리의 변경 사항을 취소하려면 restore 명령을 사용하라고 되어 있음

3. hello.txt의 수정 사항을 취소하려면 git restore 명령 다음에 한 칸 띄고 파일 이름을 쓰면 됨

```
$ git restore hello.txt
```

4. restore 명령이 정상으로 처리되면 화면에는 아무것도 나타나지 않음
cat 명령을 사용해 파일 내용을 확인해 보면 앞에서 '3'을 지우고 'three'를 추가했던 수정 내용이 사라지고 '3'이 그대로 남아 있음

```
$ cat hello.txt
```

스테이징 되돌리기 — `git restore --staged`

1. vim을 사용해서 hello2.txt를 열고 기존 내용을 삭제한 다음 대문자 A, B, C, D를 입력한 후 저장하고 편집기를 종료

```
$ vim hello2.txt
```

2. `git add` 명령으로 hello2.txt 파일을 스테이지에 올린 후 `git status` 명령으로 파일 상태를 살펴봄

```
$ git add hello2.txt  
$ git status
```

3. 상태 메시지에서 괄호 안의 내용을 보면 스테이지에서 내리려면(to unstage) `git restore --staged <file>` 명령을 사용하라고 함
4. 스테이징을 취소할 때는 `restore` 명령 뒤에 `--staged` 옵션을 붙이는데, 이 옵션만 사용하면 스테이지에 있는 모든 파일을 한꺼번에 되돌릴 수 있고, `--staged` 옵션 뒤에 파일 이름을 넣으면 해당 파일만 골라서 되돌릴 수 있음

```
$ git restore --staged hello2.txt
```

5. `git status`를 사용해 파일 상태를 확인해 보면 파일이 아직 스테이지에 올라가기 전(not staged)으로 돌아온 것을 확인할 수 있음

최신 커밋 되돌리기 — `git reset HEAD^` - (1)

1. 다시 한번 hello2.txt 문서를 수정하기 위해 vim을 열고 대문자 E를 끝에 추가한 후 저장

```
$ vim hello2.txt
```

2. `git commit` 명령을 사용해 스테이징과 커밋을 함께 실행하고 커밋 메시지는 message4로 함

```
$ git commit -am "message4"
```

3. `git log` 명령을 사용해서 제대로 커밋되었는지 확인해 보면 마지막 커밋 메시지는 message4

4. 최신 커밋을 되돌리려면 `git reset` 명령 다음에 `HEAD^`를 붙임

`HEAD^`는 현재 `HEAD`가 가리키는 브랜치의 최신 커밋을 가리킴

`git log` 명령을 실행했을 때 가장 최신 커밋에 (`HEAD -> main`) 표시가 있던 것을 기억

이렇게 되돌리면 `main`의 최신 커밋도 취소되고 스테이지에서도 내려져 작업 트리에만 파일이 남음

```
$ git reset HEAD^
```

최신 커밋 되돌리기 — `git reset HEAD^` – (2)

5. hello2.txt 파일의 커밋이 취소되고 스테이지에서도 내려졌다는 메시지가 나타남
6. `git log` 명령으로 확인해 보면 메시지가 message4인 커밋이 사라진 것을 볼 수 있음
이 방법으로 커밋을 취소하면 커밋 전에 했던 스테이징도 함께 취소됨

특정 커밋으로 되돌리기 — git reset 해시 – (1)

1. git reset 명령을 연습해 보기 위해 커밋을 몇 개 만들어야 함
빔을 사용해 hellogit 디렉터리에 rev.txt를 만들고 간단하게 영문자 'a'를 입력한 후 저장

```
$ vim rev.txt
```

2. rev.txt를 스테이지에 올린 후 커밋 메시지 R1을 붙여 커밋

```
$ git add rev.txt  
$ git commit -m "R1"
```

3. rev.txt를 한 번 더 수정해서 영문자 'b'를 추가하고, R2 메시지와 함께 커밋

```
$ vim rev.txt
```

```
$ git commit -am "R2"
```

특정 커밋으로 되돌리기 — git reset 해시 - (2)

4. 같은 방법으로 rev.txt에 영문자 'c'를 추가한 후 R3 메시지와 함께 커밋
rev.txt에 영문자 'd'를 추가한 후 R4 메시지와 함께 커밋
지금까지 커밋을 모두 4번 함
5. git log 명령을 사용해 지금까지 만든 커밋을 확인해 보세면 커밋이 4개 있고 커밋마다 커밋 해시가 함께 나타나 있음
6. 커밋 4개 가운데 R2라는 메시지가 붙은 R2 커밋으로 되돌려 보려 함
즉, R3 커밋과 R4 커밋을 삭제하고 R2 커밋을 최신 커밋으로 만들 것임
reset에서 커밋 해시를 사용해 되돌릴 때 주의할 점이 있음
예를 들어 reset A를 입력한다면 이 명령은 A 커밋을 리셋하는 것이 아니라 최근 커밋을 A로 리셋함
즉, A 커밋을 삭제하는 것이 아니라 A 커밋 이후에 만들었던 커밋을 삭제하고 A 커밋으로 이동하겠다는 의미
그래서 R3 커밋과 R4 커밋을 삭제하려면 그 이전 커밋인 R2 커밋을 최신 커밋으로 만들어야 함
최신 커밋을 가리키는 HEAD를 A로 리셋한다고 생각하면 됨
R2 커밋으로 이동하기 위해 git log 명령의 결과 화면에서 R2 커밋의 커밋 해시를 선택
마우스 오른쪽 버튼을 눌러 [Copy]를 선택

특정 커밋으로 되돌리기 — git reset 해시 - (3)

7. git reset 명령 다음에 --hard 옵션까지 입력한 후 복사한 커밋 해시를 붙여 넣고 {{ Enter }}를 누름

```
$ git reset --hard 복사한 커밋 해시
```

8. 방금 복사해서 붙인 커밋 해시 위치로 HEAD가 옮겨졌다고 나타남
즉, 방금 복사해서 붙인 커밋이 가장 최신 커밋이 된 것
9. git log 명령을 사용해서 로그 목록을 살펴봄
의도했던 대로 R4 커밋과 R3 커밋은 삭제되고 커밋 해시를 복사했던 커밋, 즉 R2 커밋이 최신 커밋이 됨
10. cat 명령을 사용해서 rev.txt 파일을 확인해 보면 내용에 'b'까지만 있음
R4 커밋은 rev.txt에 'd'를 추가한 것이고, R3 커밋은 'c'를 추가한 것, 그리고 R2 커밋은 'b'를 추가한 것이었음
'c'와 'd'를 추가했던 R4 커밋과 R3 커밋이 사라지고, R2 커밋이 최신 커밋이 되었기 때문에 'b'까지만 남은 것

커밋 변경 이력 취소하기 — `git revert` - (1)

1. 앞의 내용을 따라왔다면 `rev.txt`라는 파일에는 영문자 `a`와 `b`가 있고 커밋은 R2까지 만들어져 있음
`rev.txt` 파일을 한 번 더 수정해서 영문자 `'e'`를 추가

```
$ vim rev.txt
```

2. 수정한 `rev.txt`를 R5라는 메시지와 함께 커밋

```
$ git commit -am "R5"
```

3. `git log`를 입력해 버전을 확인해 보면 `rev.txt` 파일에 대해 R1과 R2, R5라는 버전 3개가 만들어졌음
4. 가장 최근에 커밋한 R5 버전을 취소하고, R5 직전 커밋인 R2로 되돌아가려고 함
여기에서는 취소하는 R5 버전을 삭제하지 않고 남겨 두려고 함
이럴 때는 `revert` 명령을 사용하고 명령 뒤에 취소할 버전, 즉 R5의 커밋 해시를 지정
먼저 `revert`할 R5 커밋 해시를 복사

커밋 변경 이력 취소하기 — git revert - (2)

5. revert 명령을 사용해서 R5 커밋을 취소

```
$ git revert 복사한 커밋 해시
```

6. 기본 편집기가 자동으로 나타나면서 커밋 메시지를 입력할 수 있음
커밋 메시지 맨 위에는 어떤 버전을 revert했는지 나타남
커밋을 취소하면서 남겨 둘 내용이 있다면 문서 맨 위에 입력하고 저장

7. R5 버전이 revert되었다는 간단한 메시지가 나타나는데, 실제로 버전이 어떻게 바뀌었는지 확인하기 위해 git log를 입력

```
$ git log
```

8. 로그에 R5를 revert한 새로운 커밋이 생겼고 기존의 R5 역시 사라지지 않았음
R5 버전을 지우는 대신 R5에서 변경했던 내용만 취소하고, R5를 취소했다는 커밋을 새로 만든 것

커밋 변경 이력 취소하기 — `git revert` - (3)

9. 방금 취소한 R5 커밋은 `rev.txt` 문서에 영문자 'e'를 추가한 것이었는데 R5 커밋을 취소한 것이 문서에도 반영되었는지 확인

```
$ cat rev.txt
```

10. 앞에서 추가한 'e'가 없어진 것을 볼 수 있음

이렇게 `revert` 명령을 사용하면 커밋 자체를 삭제하지 않으면서도 변경한 이력을 취소할 수 있음